



DAYANANDA SAGAR COLLEGE OF ENGINEERING
(An Autonomous Institute Affiliated to VTU, Belagavi.)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DESIGN & ANALYSIS OF ALGORITHMS

LABORATORY MANUAL

IV Semester
Course Code: 22AI43
Course Type: IPCC

Scheme: 2022-2026

Version 1.1
w.e.f. April, 2024

Editorial Committee
Design & Analysis of Algorithms Laboratory Faculty,
Dept. of AI&ML

Approved by
Dr. Vindhya P Malagi, H.O.D

Document Log

Name of the Document	Design & Analysis of Algorithms Laboratory Manual
Syllabus Scheme	2022-2026
Current Version and Date	Version 1.1, April 2024
Subject Code	22AI43
Editorial Committee	Design & Analysis of Algorithms Laboratory Faculty
Approved by	HOD, Dept. of A I & M L
Lab Faculty	Dr. Reshma S Dr. Aruna M G Prof. Ramya K
Lab Instructor	Miss. Chitrashree S

Table of Contents

S. No.	Particulars		Page No.
1	Vision and Mission of the Department		1
2	PEOs, PSOs, POs		2
3	Syllabus • Course Objectives • Course Outcomes • Conduction of Practical Examination • CO-PO-PSO Mapping		4
4	Lab Evaluation Process		8
5	CHAPTER 1	Introduction to Algorithms	9
	CHAPTER 2	Lab Programs	
	<i>Lab Session</i>	<i>Topics Covered</i>	<i>CO</i>
	Apply Brute force technique to perform sorting and searching in Arrays and Strings(2Hrs)		C05 - C06
	1	Write a Java program to implement the selection sort algorithm for sorting an array of integers.	
	2	Implement the bubble sort algorithm in Java to sort an array of strings.	
	3	Design a Java program to perform a sequential search for a given element in an array.	
	4	Write a Java program to implement the Knapsack problem using the brute force approach.	
	5	Write a Java program to implement the Travelling Salesperson problem using the brute force approach. Analyze its time complexity for different inputs.	

Implement the computational P Class problems(2Hrs)		C05 - C06
6	Write a Java program to implement the selection sort and bubble sort algorithm for sorting an array of integers.	
7	Design a Java program to perform a sequential search for a given element in an array. Analyze the time complexity of the sequential search algorithm.	
8	Write a Java program to implement the Knapsack problem using the brute force approach. Analyze its time complexity for different inputs.	
9	Write a Java program to implement the Travelling Salesperson problem using the brute force approach. Analyze its time complexity in the worst-case scenario.	
Implementation of Recursive and Non Recursive algorithms to solve numerical problems (2 Hrs)		C05 - C06
10	Write a Java program to compare the performance of recursive and non-recursive algorithms for computing Fibonacci numbers. Analyze their time complexity.	
11	Design a Java program to analyze the time of an algorithm for finding the maximum element in an array using both iterative and recursive approaches.	
12	Write a Java program to compare the performance of recursive and non-recursive algorithms for computing Factorial of a number. Analyze their time complexity.	
Demonstration of Divide and Conquer techniques and Compare time complexity with brute force techniques (2 Hrs)		C05 - C06
13	Design and implement Java Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n > 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	
14	Design and implement Java Program to sort a given set of n integer elements using MergeSort method and compute its time complexity. Run the program for varied values of n > 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	
15	Design and implement Java Program for Binary Search and compute its time complexity.	

Implement Greedy concept with graph problems (2 Hrs)			C05 - C06
16	Design and implement Java Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.		
17	Design and implement Java Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.		
18	Design and implement Java Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.		
19	Design and implement Java Program to obtain the Topological ordering of vertices in each digraph.		
Demonstrate how Dynamic Strategy better than greedy techniques with example (2 Hrs)			C05 - C06
20	Design and implement Java Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.		
21	Design and implement Java Program to find the transitive closure using Warshal's algorithm.		
22	Design and implement Java Program to solve 0/1 Knapsack problem using Dynamic Programming method.		
Illustrate the usage of Backtracking by Implementation of real time problems(2 Hrs)			C05 - C06
23	Design and implement Java Program to find a subset of a given set $S = \{s_1, s_2, \dots, s_n\}$ of n positive integers whose sum is equal to a given positive integer d.		
24	Design and implement Java Program for N Queen's problem using Backtracking.		
6	CHAPTER 3	ADDITIONAL PROGRAMS	80
7	CHAPTER 4	SAMPLE VIVA QUESTIONS	90
8		APPENDIX	92

Vision of the Department

To provide progressive education and flourish the student's ingenuity to be successful professionals impacting the society for a smarter and ethical world.

Mission of the Department

- M1:** *To adopt an engaging teaching learning process with emphasis on problem solving and programming skills.*
- M2:** *To promote additional skill development through enhanced and experiential learning.*
- M3:** *To collaborate with industries and professional bodies and make the students industry ready.*
- M4:** *To encourage innovation through multi-disciplinary research and development activities.*
- M5:** *To imbibe human values and ethics in students to make them impact the society's as responsible professionals.*

Program Educational Objectives (PEOs) of Department

After the course completion, AI&ML graduates will be able to:

- PEO1:** To practice and implement their success skills of problem solving, communication, and collaboration for providing innovative engineering solutions.
- PEO2:** Contribute their AI & ML expertise grounded in computer science and mathematics as members and leaders of professional engineering teams in multidisciplinary domains.
- PEO3:** Demonstrate lifelong learning through continued professional development and higher education in top graduate technical, research and management programs.
- PEO4:** Demonstrate a commitment to society by applying the skills and knowledge for a smarter and ethical world.

Program Specific Outcomes (PSOs)

- PSO1:** Graduates will be adept in programming and problem-solving skills for developing and managing software.
- PSO2:** Graduates will be able to identify, formulate, predict, and solve real world Problems by applying principles of Artificial Intelligence & Machine Learning.
- PSO3:** Graduates will be proficient to efficiently manage computer system resources, using cloud computing technologies.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Belagavi.)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Course Title: Design & Analysis of Algorithms

Course Code:22AI43

Course Type: IPCC

L:T:P - 3:0:2

CIE:50 SEE:50

TOTAL : 100

CREDITS: 4

Exam Duration:3 Hrs

Module No	Module Contents	Hours	COs
1	Introduction: What is an Algorithm?, Algorithm Specification , Analysis Framework), Performance Analysis: Space complexity, Time complexity . Asymptotic Notations: Big-Oh notation (O), Omega notation (Ω), Theta notation (Θ), and Little-oh notation (o), Mathematical analysis of Non-Recursive and recursive Algorithms with Examples . Brute Force Approaches: Selection Sort and Bubble Sort, Sequential Search and Brute Force String Matching, Exhaustive Search (Travelling Salesman problem and Knapsack Problem).	8 Hours	CO1-CO4
2	Divide and Conquer: General method, Binary search, Recurrence equation for divide and conquer, Finding the maximum and minimum , Merge sort, Quick sort , Strassen's matrix multiplication , Advantages and Disadvantages of divide and conquer. Decrease and Conquer Approach: Insertion Sort, , Topological Sort.	8 Hours	CO1-CO4
3	Greedy Method: General method, Coin Change Problem, Knapsack Problem, Job sequencing with deadlines . Minimum cost spanning trees: Prim's Algorithm, Kruskal's Algorithm. Single source shortest paths: Dijkstra's Algorithm . Optimal Tree problem: Huffman Trees and Codes Transform and Conquer Approach: Heaps and Heap Sort	8 Hours	CO1-CO4
4	Dynamic Programming: General method with Examples, Transitive Closure: Warshall's Algorithm, All Pairs Shortest Paths: Floyd's Algorithm, Optimal Binary Search Trees, Knapsack problem , Bellman-Ford Algorithm , Travelling Sales Person problem	8 Hours	CO1-CO4
5	Backtracking: General method , N-Queens problem, Sum of subsets problem , Graph coloring, Hamiltonian cycles . Branch and Bound: Assignment Problem, Travelling Sales Person problem, 0/1 Knapsack problem: LC Branch and Bound solution , FIFO Branch and Bound solution. NP Complete and NP-Hard problems: Basic concepts, non-deterministic algorithms, P, NP, NP-Complete, and NP-Hard classes.	8 Hours	CO1-CO4

Laboratory Session	Topics Covered	CO
Apply Brute force technique to perform sorting and searching in Arrays and Strings(2Hrs)		C05-C06
Implement the computational P Class problems(2Hrs)		C05-C06
Implementation of Recursive and Non Recursive algorithms to solve numerical problems (2 Hrs)		C05-C06
Demonstration of Divide and Conquer techniques and Compare time complexity with brute force techniques (2 Hrs)		C05-C06
Implement Greedy concept with graph problems (2 Hrs)		C05-C06
Demonstrate how Dynamic Strategy better than greedy techniques with example(2 Hrs)		C05-C06
Illustrate the usage of Backtracking by Implementation of real time problems(2 Hrs)		C05-C06

CO's	CO Statement	Hours
22AI43.1	Apply Brute Force Techniques to solve the Numerical, String and Graph problems.	10
22AI43.2	Apply Recursive Algorithms towards optimal solution for real time problems.	10
22AI43.3	Analyse the time complexity of the P Class problems.	10
22AI43.4	Analyse the time complexity of the NP and NP Hard Class problems.	10
22AI43.5	Implement and analyse the efficiency of algorithmic design techniques in solving real time problems using opensource tools.	8
22AI43.6	Write clear and concise experiment reports that detail the methods, results, and analysis of algorithm techniques.	4

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
22AI43.1	3	2	1						2	2		2	3		
22AI43.2	3	2	1						2	2		2	3		
22AI43.3	3	3	2	1					2	2		2	3		
22AI43.4	3	3	2	1					2	2		2	3		
22AI43.5	3	3	2	1					2	2		2	3		
22AI43.6	3	2	1						2	2		2	3		

LAB EVALUATION PROCESS

CIE-Practical (Conduction) :30

Min 12

WEEK WISE EVALUATION OF EACH PROGRAM

ACTIVITY	MARKS
Execution	20
Observation Book + Record	10
TOTAL	30

CIE-Practical (Test) :20 Min 08

INTERNAL ASSESSMENT EVALUATION (End of Semester)		
S.No	ACTIVITY	MARKS
01	Write-up	10
02	Conduction	35
03	Viva Voce	05
	Total	50 (Reduced to 20 min 08)

Total CIE-Practical :50 Min 20 (Reduced to 20 Min 08)

FINAL INTERNAL ASSESSMENT CALCULATION		
S.No	ACTIVITY	MARKS
01	Average of weekly entries	30
02	Internal Assessment reduced to	20
	Total	50

Reduce overall lab marks from 50 to 20. The minimum marks to be secured in CIE of the LAB to appear for SEE of IPCC shall be the 8 marks [40% of maximum marks (20)].

CIE-Practical (CONTINUOUS INTERNAL EVALUATION) : 30

Continuous Evaluation done every week.

Rubrics	MARKS
Observation	5
Record	5
Conduction	5
Viva	5
Attendance	10
TOTAL	30

Rubrics for Continuous Internal Evaluation :

Table 1: Rubrics for lab CIE

Max Marks: 20 (excluding attendance)

Evaluation / Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
Observation (Inferring to the programs in the lab & documenting) (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, inference not. Completed on time	Complete programs with comments including analysis and inference provided and completed on time
Conduction (05 M)	Executed the program without understanding, rectified the errors and lack of conceptual understanding.	Successfully executed the program with basic understanding of the program	Successfully executed the program with good understanding of the program (function level)	Successfully executed the program with detailed understanding of the program (line by line)
Records (05 M)	Incomplete programs and missing analysis part, late submission	Complete programs with comments however analysis and inference not presented.	Complete programs with comments including analysis, however inference not presented appropriately. Submitted on time.	Complete programs with comments including analysis and inference provided & submitted on time
Viva (05 M)	1 question out of 5 viva questions answered on a one-on-one basis	Basic understanding on the programs and 2 or 3 questions out of 5 viva questions answered on a one-on-one basis	Good understanding on the programs and 4 questions out of 5 viva questions answered on a one-on-one basis	Programs well understood and all 5 questions answered on a one-on-one basis
Score	Score 1 (S1)	Score 2 (S2)	Score 3 (S3)	Score 4 (S4)
Total Score				S1+ S2+S3+S4

In the conduction criteria of CIE (During internal lab exams), the following Rubrics are considered.

- Syntax
- Logic
- Correctness
- Completeness of the program

Table 2: Rubrics for lab programs conduction component

Max Marks:5

Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
Syntax (5) Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
Correctness (5) Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
Total	Score /4			

Rubrics for write up and execution of Internal Examination

TOPIC	MARKS
Write up	10
Execution	35
Viva	5
TOTAL	50

Table 3: Rubrics for write up Internal Examination

Max Marks:10

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3-5)	Good (6-8)	Excellent (9-10)
W R I T E - U P	Syntax (5) Ability to understand and follow the rules of the program	Program doesn't compile and has errors and warnings	Program compiles, but contains warnings that lead to incorrect logic	Program compiles and it is free from errors but may contain non-standard usage of programming	Program compiles without errors and follows the standards of programming
	Logic (5) Ability to identify conditions, control flow and data structures for the given program	Incorrect program logic such as infinite loops	Program logic is correct without any infinite loops, however all possible boundary cases not addressed	Program logic is correct with few boundary conditions	Program logic is correct with all conditions and cases coded with no ambiguity
	Total	Score*1			

Table 3: Rubrics for execution of Internal Examination

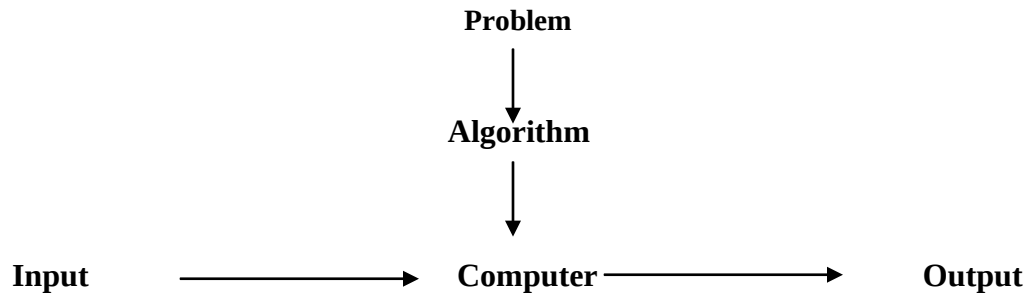
Max Marks:35

	Rubrics	Not Satisfactory (1-2)	Satisfactory (3)	Good (4)	Excellent (5)
E X E C U T I O N	Correctness (5) Ability to code algorithms that produce correct appropriate output for the given program	Incorrect output or inappropriate output for any given test case	Program gives correct output for most inputs but not for all test cases	Program gives correct output for most test cases	Program gives correct output for all test cases
	Completeness (5) Ability to apply critical analysis to check for the completeness of the program	Program shows little or no recognition on different cases leading to incompleteness	Program shows evidence of case analysis and missing significant cases or missed cases	Most of the requirements of the program met however all possible cases not handled	All requirements of the program met and all possible cases handled
	Total	Score*3.5			

CHAPTER 1

INTRODUCTION TO ALGORITHMS

An algorithm is a sequence of finite number of unambiguous instructions for solving a problem, i.e for obtaining a required output for any legitimate input in a finite amount of time.



- The non ambiguity requirement for each step of an algorithm.
- The range of inputs has to be specified carefully.
- The same algorithm can be represented in several different ways.
- Several algorithms for solving the same problem may exist.
- Algorithm for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

Properties Of Algorithms

1. **Input** – an algorithm has zero or more inputs.
2. **Output** – an algorithm has one or more outputs.
3. **Finiteness** – an algorithm must terminate after a finite number of steps.
4. **Definiteness** – each step must be precisely defined.
5. **Effectiveness** – all operations can be carried out exactly in finite time (lesser amount of time).

Algorithm Paradigms

Often there are large collections of problems that can be solved using the same general techniques or paradigms. A few of the most common are described below:

Brute Force: A straightforward approach to solving a problem based on the problem statement and concepts involved. Brute force algorithms are rarely efficient. Example algorithms include:

- Bubble sort.
- Computing the sum of n numbers by direct addition.
- Standard matrix multiplication.
- Linear search.

Divide And Conquer:

Perhaps the most famous algorithm paradigm, divide and conquer is based on partitioning the problem into two or more smaller sub-problems, solving them and combining the sub-problem solutions into a solution for the original problem. Example algorithms include:

- Binary Search, Merge sort and quick sort.
- The Fast Fourier Transform (FFT).
- Strassen's matrix multiplication.

Greedy Algorithms:

Greedy algorithms always make the choice that seems best at the moment.

This locally optimal choice is made with the hope that it leads to a globally optimal solution.

Some greedy algorithms may not be guaranteed to always produce an optimal solution.

Greedy algorithms are often applied to combinatorial optimization problems.

Given an instance of the problems, there is a set of candidates or feasible solutions that satisfy the constraints of the problem.

For each feasible solution there is a value determined by an objective function. An optimal solution minimizes (or maximizes) the value of objective function. Example algorithms include:

- Kruskal's and Prim's minimal spanning tree algorithms.
- Dijkstra's single source shortest path algorithm.
- Huffman coding.

Dynamic Programming:

A nutshell definition of dynamic programming is difficult, but to summarize, problems which lend themselves to a dynamic programming attack have the following characteristics: We have to search over a large space for an optimal solution. The optimal solution can be expressed in terms of optimal solution to sub-problem.

The number of sub-problems that must be solved is small.

Dynamic programming algorithms have the following features:

- A recurrence that is implemented iteratively.
- A table, built to support the iteration.
- Tracing through the table to find the optimal solution.
- Example algorithms include Efficient Fibonacci number computation, knapsack problems, The Floyd's and Warshall's algorithm.

Measuring Time Efficiency:

long startTime = System.nanoTime();

records the start time before the binary search operation.

long endTime = System.nanoTime();

records the end time after the binary search operation.

long duration = endTime - startTime;

calculates the duration of the binary search in nanoseconds.

CHAPTER 2

Program 1

Write a Java program to implement the selection sort algorithm for sorting an array of integers use Bruteforce Technique.

Program 2

Implement the bubble sort algorithm in Java to sort an array of strings use Bruteforce Technique .

```
.  
package edu.dsce;  
  
public class BubbleSort {  
    public static void main(String[] args) {  
        String[] arr = {"banana", "apple", "grape", "orange",  
"pineapple"};  
        bubbleSort(arr);  
        System.out.println("Sorted array:");  
        printArray(arr);  
    }  
  
    public static void bubbleSort(String[] arr) {  
        int n = arr.length;  
        for (int i = 0; i < n-1; i++) {  
            for (int j = 0; j < n-i-1; j++) {  
                if (arr[j].compareTo(arr[j+1]) > 0) {  
                    // Swap arr[j] and arr[j+1]  
                    String temp = arr[j];  
                    arr[j] = arr[j+1];  
                    arr[j+1] = temp;  
                }  
            }  
        }  
    }  
  
    public static void printArray(String[] arr) {  
        for (String s : arr) {  
            System.out.print(s + " ");  
        }  
        System.out.println();  
    }  
}
```

Program 3

Design a Java program to perform a sequential search for a given element in an array use Brute force Technique.

```
package edu.dsce;

import java.util.Scanner;

public class SequentialSearch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input array elements
        System.out.print("Enter the number of elements in the array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter the elements of the array:");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        // Input search key
        System.out.print("Enter the element to search for: ");
        int searchKey = scanner.nextInt();

        // Perform sequential search
        int index = sequentialSearch(arr, searchKey);
        if (index != -1) {
            System.out.println("Element " + searchKey + " found at index " + index);
        } else {
            System.out.println("Element " + searchKey + " not found in the array");
        }

        scanner.close();
    }

    public static int sequentialSearch(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                return i; // Element found, return its index
            }
        }
        return -1; // Element not found
    }
}
```

Program 4

Develop a Java program to perform brute force string matching for a given pattern in a text string array use Brute force Technique.

```
.
package edu.dsce;

import java.util.Scanner;

public class BruteForceStringMatching {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input text and pattern
        System.out.print("Enter the text: ");
        String text = scanner.nextLine();
        System.out.print("Enter the pattern to search for: ");
        String pattern = scanner.nextLine();

        // Perform brute force string matching
        int index = bruteForceStringMatch(text, pattern);

        // Output result
        if (index != -1) {
            System.out.println("Pattern found at index " + index + " in the text.");
        } else {
            System.out.println("Pattern not found in the text.");
        } scanner.close();
    }

    public static int bruteForceStringMatch(String text, String pattern)
    {
        int n = text.length();
        int m = pattern.length();

        for (int i = 0; i <= n - m; i++) {
            int j;
            for (j = 0; j < m; j++) {
                if (text.charAt(i + j) != pattern.charAt(j)) {
                    break;
                }
            }
            if (j == m) { // Match found
                return i;
            }
        }
        return -1; // Pattern not found
    }
}
```

Program 5

Write a Java program to implement the Knapsack problem using the brute force approach array use Brute force Technique.

```
import java.util.Scanner;

public class KnapsackBruteForce {

    // Helper method to calculate the maximum value using brute force
    private static int knapsack(int[] weights, int[] values, int n, int
maxWeight) {
        int maxValue = 0;
        int numberOfSubsets = (int) Math.pow(2, n);

        for (int i = 0; i < numberOfSubsets; i++) {
            int totalWeight = 0;
            int totalValue = 0;
            for (int j = 0; j < n; j++) {
                if ((i & (1 << j)) != 0) {
                    totalWeight += weights[j];
                    totalValue += values[j];
                }
            }
            if (totalWeight <= maxWeight && totalValue > maxValue) {
                maxValue = totalValue;
            }
        }
        return maxValue;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read the number of items
        System.out.print("Enter the number of items: ");
        int n = scanner.nextInt();

        // Read the weights and values of the items
        int[] weights = new int[n];
        int[] values = new int[n];
        System.out.println("Enter the weights of the items:");
        for (int i = 0; i < n; i++) {
            weights[i] = scanner.nextInt();
        }
        System.out.println("Enter the values of the items:");
        for (int i = 0; i < n; i++) {
            values[i] = scanner.nextInt();
        }
    }
}
```

```

    }

    // Read the maximum weight capacity of the knapsack
    System.out.print("Enter the maximum weight capacity of the
knapsack: ");
    int maxWeight = scanner.nextInt();

    // Calculate the maximum value
    int maxValue = knapsack(weights, values, n, maxWeight);
    System.out.println("The maximum value that can be obtained is: "
+ maxValue);

    scanner.close();
}
}

```

Enter the number of items: 4

Enter the weights of the items:

2 3 4 5

Enter the values of the items:

3 4 5 6

Enter the maximum weight capacity of the knapsack: 5

The maximum value that can be obtained is: 7

Program 6

Write a Java program to implement the Travelling Salesperson problem using the brute force approach. Analyze its time complexity for different inputs.

```
package edu.dsce;

import java.util.*;

public class TSPBruteForce {
    static int[][] graph;
    static int numCities;
    static int[] tour;
    static int minDistance = Integer.MAX_VALUE;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input number of cities
        System.out.print("Enter the number of cities: ");
        numCities = scanner.nextInt();

        // Input the adjacency matrix representing the graph
        graph = new int[numCities][numCities];
        System.out.println("Enter the adjacency matrix representing the graph:");
        for (int i = 0; i < numCities; i++) {
            for (int j = 0; j < numCities; j++) {
                graph[i][j] = scanner.nextInt();
            }
        }

        // Initialize tour array to store the path
        tour = new int[numCities];
        for (int i = 0; i < numCities; i++) {
            tour[i] = i;
        }

        // Start the brute force search
        bruteForceTSP(1);

        // Output the minimum distance
        System.out.println("Minimum distance for TSP: " + minDistance);

        scanner.close();
    }

    public static void bruteForceTSP(int start) {
```

```

        if (start == numCities) {
            int distance = calculateDistance();
            minDistance = Math.min(minDistance, distance);
            return;
        }

        for (int i = start; i < numCities; i++) {
            swap(start, i);
            bruteForceTSP(start + 1);
            swap(start, i);
        }
    }

    public static void swap(int i, int j) {
        int temp = tour[i];
        tour[i] = tour[j];
        tour[j] = temp;
    }

    public static int calculateDistance() {
        int distance = 0;
        for (int i = 0; i < numCities - 1; i++) {
            distance += graph[tour[i]][tour[i + 1]];
        }
        distance += graph[tour[numCities - 1]][tour[0]]; // Return to the
starting city
        return distance;
    }
}

```

Enter the number of cities: 4

Enter the cost matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Minimum cost: 80

Path: 0 1 3 2 0

Program 7

Write a Java program to compare the performance of recursive and non-recursive algorithms for computing Fibonacci numbers. Analyze their time complexity.

```
import java.util.Scanner;

public class FibonacciComparison {

    // Recursive method to compute Fibonacci number
    public static long fibonacciRecursive(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
    }

    // Non-recursive (iterative) method to compute Fibonacci number
    public static long fibonacciIterative(int n) {
        if (n <= 1) {
            return n;
        }
        long prev2 = 0, prev1 = 1, current = 1;
        for (int i = 2; i <= n; i++) {
            current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }
        return current;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```

System.out.print("Enter the value of n: ");
int n = scanner.nextInt();

// Measure time for recursive method
long startRecursive = System.nanoTime();
long resultRecursive = fibonacciRecursive(n);
long endRecursive = System.nanoTime();
long timeRecursive = endRecursive - startRecursive;

System.out.println("Fibonacci number (recursive) for n=" + n + "
is: " + resultRecursive);
System.out.println("Time taken by recursive method: " +
timeRecursive + " nanoseconds");

// Measure time for non-recursive (iterative) method
long startIterative = System.nanoTime();
long resultIterative = fibonacciIterative(n);
long endIterative = System.nanoTime();
long timeIterative = endIterative - startIterative;

System.out.println("Fibonacci number (iterative) for n=" + n + "
is: " + resultIterative);
System.out.println("Time taken by iterative method: " +
timeIterative + " nanoseconds");

scanner.close();
}
}

```

```

Enter the Fibonacci term to compute: 40
Fibonacci term 40 (Recursive): 102334155
Time taken (Recursive): 41209530 nanoseconds
Fibonacci term 40 (Iterative): 102334155
Time taken (Iterative): 1849 nanoseconds

```

Program 8

Design a Java program to analyze the time of an algorithm for finding the maximum element in an array using both iterative and recursive approaches.

Time Complexity Analysis

- **Iterative Method:**

Time Complexity: $O(n)O(n)$

The iterative method scans through the array once, comparing each element to find the maximum. Therefore, the time complexity is linear with respect to the size of the array.

Space Complexity: $O(1)O(1)$

The space complexity is constant because the method uses only a fixed amount of additional space, regardless of the size of the array.

- **Recursive Method:**

Time Complexity: $O(n)O(n)$

The recursive method also examines each element once to find the maximum, leading to a linear time complexity.

Space Complexity: $O(n)O(n)$

The space complexity is linear due to the call stack. Each recursive call adds a new frame to the stack, resulting in nn frames for an array of size nn .

Explanation

The **iterative method** is generally more efficient in terms of space because it uses constant space regardless of the input size.

- The **recursive method** can be more elegant and easier to understand for some people, but it has a higher space complexity due to the call stack.

```
import java.util.Random;
import java.util.Scanner;

public class MaxElementComparison {

    // Iterative method to find the maximum element in an array
    public static int findMaxIterative(int[] array) {
        int max = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max) {
                max = array[i];
            }
        }
        return max;
    }
}
```

```

}

// Recursive method to find the maximum element in an array
public static int findMaxRecursive(int[] array, int index) {
    if (index == array.length - 1) {
        return array[index];
    }
    int maxInRest = findMaxRecursive(array, index + 1);
    return Math.max(array[index], maxInRest);
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("Enter the size of the array: ");
    int size = scanner.nextInt();
    int[] array = new int[size];

    // Fill the array with random integers
    Random random = new Random();
    for (int i = 0; i < size; i++) { array[i] =
random.nextInt(10000); // Random integers between 0 and 9999
    }

    // Measure time for iterative method
    long startIterative = System.nanoTime();
    int maxIterative = findMaxIterative(array);
    long endIterative = System.nanoTime();
    long timeIterative = endIterative - startIterative;

    System.out.println("Maximum element (iterative) is: " + maxIterative);
    System.out.println("Time taken by iterative method: " + timeIterative
+ " nanoseconds");
}

```

```

        // Measure time for recursive method
        long startRecursive = System.nanoTime();
        int maxRecursive = findMaxRecursive(array, 0);
        long endRecursive = System.nanoTime();
        long timeRecursive = endRecursive - startRecursive;

        System.out.println("Maximum element (recursive) is: " + maxRecursive);
        System.out.println("Time taken by recursive method: " + timeRecursive +
" nanoseconds");

        scanner.close();
    }
}

```

Output :

```

Enter the size of the array: 10
Enter the elements of the array:
4 7 2 9 5 8 3 1 6 10
Maximum element (Iterative): 10
Time taken (Iterative): 2015 nanoseconds
Maximum element (Recursive): 10
Time taken (Recursive): 4322 nanoseconds

```

Program 9

Write a Java program to compare the performance of recursive and non-recursive algorithms for computing Factorial of a number. Analyze their time complexity

Time Complexity Analysis

2. Recursive Method:

- Time Complexity: $O(n)O(n)$
 - The recursive method computes the factorial by making n recursive calls, each performing a constant amount of work. Therefore, the time complexity is linear.
- Space Complexity: $O(n)O(n)$
 - The space complexity is also linear due to the call stack, which grows with each recursive call until it reaches the base case.

3. Iterative Method:

- Time Complexity: $O(n)O(n)$
 - The iterative method computes the factorial by iterating from 2 to n , performing a constant amount of work in each iteration. Hence, the time complexity is linear.
- Space Complexity: $O(1)O(1)$
 - The space complexity is constant because the method uses a fixed amount of additional space regardless of the input size.

```
import java.util.Scanner;
```

```
public class FactorialComparison {
```

```
    // Recursive method to compute factorial
```

```
    public static long factorialRecursive(int n) {
```

```
        if (n == 0 || n == 1) {
```

```
            return 1;
```

```
        }
```

```
        return n * factorialRecursive(n - 1);
```

```
    }
```

```
    // Iterative method to compute factorial
```

```
    public static long factorialIterative(int n) {
```

```
        long result = 1;
```

```
        for (int i = 2; i <= n; i++) {
```

```
            result *= i;
```

```
        }
```

```

        return result;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int n = scanner.nextInt();

        // Measure time for recursive method
        long startRecursive = System.nanoTime();
        long resultRecursive = factorialRecursive(n);
        long endRecursive = System.nanoTime();
        long timeRecursive = endRecursive - startRecursive;

        System.out.println("Factorial (recursive) of " + n + " is: " +
resultRecursive);

        System.out.println("Time taken by recursive method: " +
timeRecursive + " nanoseconds");

        // Measure time for iterative method
        long startIterative = System.nanoTime();
        long resultIterative = factorialIterative(n);
        long endIterative = System.nanoTime();
        long timeIterative = endIterative - startIterative;

        System.out.println("Factorial (iterative) of " + n + " is: " +
resultIterative);

        System.out.println("Time taken by iterative method: " +
timeIterative + " nanoseconds");

        scanner.close();
    }

```

} Program 10

Sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of $n > 5000$ and record the time taken to sort. Plot a graph of the time taken versus non graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate using Java how the divide and conquer method works along with its time complexity analysis: worst case, average case and best case.

QUICKSORT

Quick sort is one of the sorting algorithms working on the Divide and Conquer strategy. In Quick sort the given array elements is divided into two parts based on pivot(a key element) . Pivot element partitions the array into three sub arrays such that

- The left sub array contains all elements which are less than or equal to the pivot.
- The middle sub arrays contains pivot.
- The right sub array contains all elements which are greater than or equal to the pivot. The innermost loop is so efficient that it runs faster than merge sort and heap sort.

Time Complexity Analysis:

- Best case performance is $\Omega(n \log n)$
- Average case performance is $\Theta(1.38 * n \log n)$
- Worst case Performance is $O(n^2)$

```
import
java.util.Scanner;
import
java.util.Arrays;
import
java.util.Random;
public class QuickSortComplexity
{
    static final int MAX =
    200000; static int[] a;
    static Random random = new
    Random(); public static void
    main(String[] args)
    {
        Scanner sc = new
        Scanner(System.in);
        System.out.print("Enter Max array
        size: ");
        int n = sc.nextInt();
        a= new int[n];
        System.out.println("Enter the array
        elements: ");
        for (int i = 0; i < n; ++i)
            a[i] = random.nextInt(1000);
        System.out.println("---The Best Case and AVG Case
```

```

        Scenario---"); TimeComplexity(n);
        System.out.println("---The Worst Case
        Scenario---"); Arrays.sort (a);
        TimeComplexity(n);
        sc.close();
    }
    public static void TimeComplexity(int n)
    {
        //remove comment-line to print input
        //System.out.println("Input Array:");
        //for (int i = 0; i < n; i++)
        //System.out.print(a[i] +
        " "); long startTime=0;
        startTime=
        System.nanoTime();
        QuickSortAlgorithm(0,  n
        - 1); long stopTime=0;
        stopTime = System.nanoTime();
        long elapsedTime = stopTime - startTime;

        //remove comment-line to print sorted array
        //System.out.println("\nSorted Array:");
        //for (int i = 0; i < n; i++)
        //System.out.print(a[i] +
        " ");
        System.out.println();
        System.out.println("Time Complexity in ms for n=" + n
        + " is: " + (double) elapsedTime / 1000000);
    }
    public static void QuickSortAlgorithm(int p, int r)
    {
        int i, j, temp,
        pivot; if (p < r)
        {
            i = p;
            j = r + 1;
            pivot = a[p]; // mark first element
            as pivot while (true)
            {
                i++;
                while (a[i] < pivot && i
                < r) i++;
                j--;
                while (a[j] >
                pivot) j--;
                if (i < j)
                {
                    temp=a[i];
                    a[i]=a[j];
                    a[j]=temp;
                }
                else
                break;// partition over

```

```

        a[p]=a[j];
        a[j]=pivot;
QuickSortAlgorithm(p, j - 1);
QuickSortAlgorithm(j + 1, r);
    }
}

```

OUTPUT

NOTE: (Plot a Graph for all the cases taking the inputs for min 4-5 values)

RUN 1	RUN 2	RUN 3
<pre> Enter Max array size: 3 Enter the array elements: ---The Best Case and AVG Case Scenario--- Time Complexity in ms for n=3 is: 0.003021 ---The Worst Case Scenario--- Time Complexity in ms for n=3 is: 0.00151 </pre>		

Program 11

Sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of $n > 5000$, and record the time taken to sort. Plot a graph of the time taken versus non graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate using Java how the divide and conquer method works along with its time complexity analysis: worst case, average case and best case.

MERGE SORT

Merge sort is a perfect example of a successful application of the divide-and-conquer technique. The steps followed in merge sort are:

- Merge sort works by dividing the given array into two sub arrays of equal size
- The sub arrays are sorted independently using recursion
- The sorted sub array are then merged to get the solution for the original array
- In merge sort given set of elements (n) split into sets $a[1].....a[n/2]$ and $a[n/2+1].....a[n]$.
- Each set is individually sorted and resulting sorted sequence merged to produce a single sorted sequence of n elements

Time Complexity Analysis:

Best/Average/Worst case performance is $\Theta(n \log n)$

Solution

```
import
java.util.Random;
import
java.util.Scanner;
public class
MergeSort
{
    static final int MAX =
    200000; static int[] a =
    new int[MAX];
    public static void main(String[] args)
    {
        Scanner sc = new
        Scanner(System.in);
```

```

        System.out.print("Enter Max array
size: "); int n = sc.nextInt();
Random random = new Random();
//System.out.println("Enter the array
elements: "); for (int i = 0; i < n; i++)
{
    a[i] = random.nextInt(1000);
    //System.out.print(a[i] + " ");
}
long startTime =
System.nanoTime();
MergeSortAlgorithm(0, n - 1);
long stopTime = System.nanoTime();
long elapsedTime = stopTime - startTime;
System.out.println("---The Best Case, AVG Case
and Worst CaseScenario---");
System.out.println("Time Complexity (ms) for n = " +
n + " is
: " + (double) elapsedTime / 1000000);
//System.out.println("Sorted Array (Merge Sort):");
//for (int i = 0; i < n; i++)
//System.out.print(a[i] +
" "); sc.close();
}

public static void MergeSortAlgorithm(int low, int high)
{
    int mid;
    if (low < high)
    {
        mid = (low + high) / 2;
        MergeSortAlgorithm(low,
mid); MergeSortAlgorithm(mid
+ 1, high); Merge(low, mid,
high);
    }
}

```

```

    }
}
public static void Merge(int low, int mid, int high)
{
    int[] b = new
    int[MAX];    int i,
    h, j, k;
    h = i =
    low;
    j = mid +
    1;
    while ((h <= mid) && (j <= high))
        if (a[h] < a[j])
            b[i++] = a[h++];
        else
            b[i++] = a[j++];
    if (h > mid)
        for (k = j; k <= high; k++)
            b[i++] = a[k];
    else
        for (k = h; k <= mid; k++)
            b[i++] = a[k];
    for (k = low; k <= high; k++)
        a[k] = b[k];
}
}

```

OUTPUT

(Plot a Graph for all the cases taking the inputs for min 8-10 values)

RUN 1	RUN 2	RUN 3
<pre> Enter Max array size: 10 ---The Best Case, AVG Case and Worst CaseScenario--- Time Complexity (ms) for n = 10 is : 1.824471 </pre>		

Program 12

Design and implement Java Program for Binary Search and compute its time complexity.

Binary Search Algorithm

search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing the portion of the list that could contain the item in half until you've narrowed the possible locations to just one.

Here is how the algorithm works:

1. Start with the middle element of the array.
2. If the target value is equal to the middle element, you have found the value.
3. If the target value is less than the middle element, repeat the search on the left half of the array.
4. If the target value is greater than the middle element, repeat the search on the right half of the array.
5. If the range reduces to zero, the target is not in the array.

Time Complexity

The time complexity of binary search is $O(\log n)$, where n is the number of elements in the array. This is because with each step, the algorithm divides the range of elements in half.

```
import java.util.Arrays;
import java.util.Scanner;

public class BinarySearch {

    // Binary search function
    public static int binarySearch(int[] array, int target) {
        int left = 0;
        int right = array.length - 1;

        while (left <= right) {
            int middle = left + (right - left) / 2;

            // Check if target is present at mid
            if (array[middle] == target) {
                return middle;
            }
        }
    }
}
```

```

        // If target greater, ignore left half
        if (array[middle] < target) {
            left = middle + 1;
        }
        // If target is smaller, ignore right half
        else {
            right = middle - 1;
        }
    }

    // Target not found
    return -1;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Read array length
    System.out.print("Enter the number of elements in the array: ");
    int n = scanner.nextInt();

    // Read array elements
    int[] array = new int[n];
    System.out.println("Enter the elements of the array:");
    for (int i = 0; i < n; i++) {
        array[i] = scanner.nextInt();
    }

    // Ensure the array is sorted
    Arrays.sort(array);

    // Read target value
    System.out.print("Enter the target value to search: ");
    int target = scanner.nextInt();

    // Measure start time
    long startTime = System.nanoTime();

    // Perform binary search
    int result = binarySearch(array, target);

    // Measure end time
    long endTime = System.nanoTime();
    long duration = endTime - startTime;

    // Output results
    if (result == -1) {
        System.out.println("Element not present in array");
    } else {
        System.out.println("Element found at index " + result);
    }

    // Output time taken
    System.out.println("Time taken for binary search: " + duration + "
nanoseconds");

    scanner.close();
}
}

```

Program 13

Implement in Java, the 0/1 Knapsack problem using Greedy method.

Knapsack problem

Given n objects, their weights w_i and profits p_i , where $i=1$ to n , and the capacity of the knapsack is W , the objective is select the objects for the knapsack such that the maximum profit is earned and the weight of the objects chosen shouldn't exceed the capacity of the given knapsack .

```
import java.util.Scanner;
class KObject
{
    // Knapsack object details
    float w, p, r;
}
public class KnapsackGreedy
{
    static final int MAX = 21; // max. no. of objects
    static int n; // no. of objects
    static float M; // capacity of Knapsack

    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of objects: ");
        n = sc.nextInt();
        KObject[] obj = new KObject[n];
        for(int i = 0; i<n;i++)
            obj[i] = new KObject();// allocate memory for members
        ReadObjects(obj);
        Knapsack(obj);
        sc.close();
    }
    static void ReadObjects(KObject obj[])
    {
        KObject temp = new KObject();
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the max capacity of knapsack: ");
        M = sc.nextFloat();
        System.out.println("Enter Weights: ");
        for (int i = 0; i < n;i++)
            obj[i].w = sc.nextFloat();
        System.out.println("Enter Profits: ");
        for (int i = 0; i < n;i++)
            obj[i].p = sc.nextFloat();
        for (int i = 0; i < n;i++)
            obj[i].r = obj[i].p / obj[i].w;
        for(int i = 0; i<n-1;i++)
            // sort objects in descending order, based on p/w ratio
            for(int j=0; j<n-1-i;j++)
                if(obj[j].r < obj[j+1].r)
                {
                    temp = obj[j];
```

```

        obj[j] = obj[j+1];
        obj[j+1] = temp;
    }
    sc.close();
}
static void Knapsack(KObject kobj[])
{
    float x[] = new float[MAX];
    float totalprofit;
    int i;
    float U; // U place holder for M
    U = M;
    totalprofit = 0;
    for (i = 0; i < n; i++)
        x[i] = 0;
    for (i = 0; i < n; i++)
    {
        if (kobj[i].w > U)
            break;
        else
        {
            x[i] = 1;
            totalprofit = totalprofit + kobj[i].p;
            U = U - kobj[i].w;
        }
    }
    System.out.println("i = " + i);
    if (i < n)
        x[i] = U / kobj[i].w;
    totalprofit = totalprofit + (x[i] * kobj[i].p);
    System.out.println("The Solution vector, x[]: ");
    for (i = 0; i < n; i++)
        System.out.print(x[i] + " ");
    System.out.println("\nTotal profit is = " + totalprofit);
}
}

```

OUTPUT

RUN 1	RUN 2	RUN 3
Enter number of objects: 3 Enter the max capacity of knapsack: 20 Enter Weights: 18 15 10 Enter Profits: 25 24 15 i = 1 The Solution vector, x[]: 1.0 0.5 0.0 Total profit is = 31.5		

Program 14

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm. Write the program in Java.

Solution :

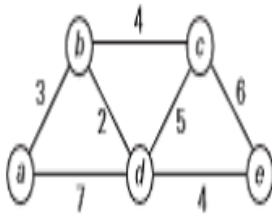
```
import java.util.*;
public class DijkstrasClass
{
    final static int MAX = 20;
    final static int infinity = 9999;
    static int n; // No. of vertices of G
    static int a[][]; // Cost matrix
    static Scanner sc = new Scanner(System.in);
    public static void main(String[] args)
    {
        ReadMatrix();
        int s = 0; // starting vertex
        System.out.println("Enter starting vertex: ");
        s = sc.nextInt();
        Dijkstras(s); // find shortest path
    }
    static void ReadMatrix()
    {
        a = new int[MAX][MAX];
        System.out.println("Enter the number of vertices:");
        n = sc.nextInt();
        System.out.println("Enter the cost adjacency matrix:");
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                a[i][j] = sc.nextInt();
    }
    static void Dijkstras(int s)
    {
        int S[] = new int[MAX];
        int d[] = new int[MAX];
        int u, v, i;
        for (i = 1; i <= n; i++)
        {
            S[i] = 0;
            d[i] = a[s][i];
        }
        S[s] = 1;
        d[s] = 1;
        i = 2;
        while (i <= n)
        {
            u = Extract_Min(S, d);
            S[u] = 1;
            i++;
            for (v = 1; v <= n; v++)
            {
                if (((d[u] + a[u][v] < d[v]) && (S[v] == 0)))
                    d[v] = d[u] + a[u][v];
            }
        }
    }
}
```

```

        for (i = 1; i <= n; i++)
        if (i != s)
        //System.out.println(i + ":" + d[i]);
        System.out.println("source :"+s+"\t destination :"+ i+"\t MinCost
        is :"+ d[i]+"\t");
    }
    static int Extract_Min(int S[], int d[])
    {
        int i, j = 1, min;
        min =infinity;
        for (i = 1; i <= n; i++)
        {
            if ((d[i] < min) && (S[i] == 0))
            {
                min = d[i];
                j = i;
            }
        }
        return (j);
    }
}

```

OUTPUT:



RUN 1	RUN 2	RUN 3
<pre> Enter the number of vertices: 5 Enter the cost adjacency matrix: 0 3 999 7 999 3 0 4 2 999 999 4 0 5 6 7 2 5 0 4 999 999 6 4 0 Enter starting vertex: 1 source :1 destination :2 MinCost is :3 source :1 destination :3 MinCost is :7 source :1 destination :4 MinCost is :5 source :1 destination :5 MinCost is :9 </pre>		

Program 15

Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.

Solution :

```
import java.util.Scanner;
class Kruskals
{
    public static void main(String [] args)
    {
        Compute c = new Compute();
        c.ReadFromConsole();
        c.Kruskal();
    }
}
class Compute
{
    private static int n;
    int [][]cost;
    int []parent;
    public void ReadFromConsole()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of nodes");
        n= sc.nextInt();
        cost = new int[n+2][n+2] ;
        parent = new int[n+2];
        System.out.println("Enter cost matrix");
        for(int i=1;i<=n;i++)
        {
            parent[i]=0;
            for(int j=1;j<=n;j++)
            {
                cost[i][j] = sc.nextInt();
                if (cost[i][j]==0) cost[i][j] = 999;
            }
        }
        sc.close();
    }
    public void Kruskal()
    {
        int i,j,a=0,b=0,u=0,v=0,min,mincost=0,ne=1;
        System.out.println("The edges of Minimum Cost Spanning Tree are");
        while(ne<n)
        {
            for(i=1,min=999;i<=n;i++)
            for(j=1;j<= n;j++)
            if(cost[i][j] < min)
            {
                min=cost[i][j];
                a=u=i;
                b=v=j;
            }
        }
    }
}
```

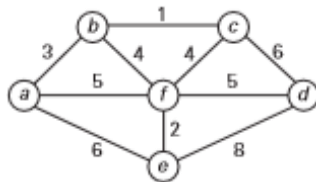
```

        u=find(u);
        v=find(v);
        if(uni(u,v)==1)
        {
            System.out.println(ne++ + " edge(" + a + "," + b + ")="
                                + min);
            mincost +=min;
        }
        cost[a][b]= cost[b][a] = 999;
    }
    System.out.println("Minimum cost = " + mincost);
}
public int find(int i)
{
    while(parent[i]!=0)
        i=parent[i];
    return i;
}

public int uni(int i,int j)
{
    if (i != j)
    {
        parent[j] = i;
        return 1;
    }
    return 0;
}
}

```

OUTPUT



RUN 1

```

Enter number of nodes
6
Enter cost matrix
0 3 999 999 6 5
3 0 1 999 999 4
999 1 0 6 999 4
999 999 6 0 8 5
6 999 999 8 0 2
5 4 4 5 2 0
The edges of Minimum Cost Spanning Tree are
1 edge(2,3) = 1
2 edge(5,6) = 2
3 edge(1,2) = 3
4 edge(2,6) = 4
5 edge(4,6) = 5
Minimum cost = 15

```

RUN 2

RUN 3

Program 16

9. Find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

Solution :

```
import java.util.Scanner;
public class PrimsClass
{
    final static int MAX = 20;
    static int n; // No. of vertices of G
    static int cost[][]; // Cost matrix
    static Scanner sc = new Scanner(System.in);
    public static void main(String[] args)
    {
        ReadMatrix();
        Prims();
    }
    static void ReadMatrix()
    {
        int i, j;
        cost = new int[MAX][MAX];
        System.out.println("Enter the number of nodes:");
        n = sc.nextInt();
        System.out.println("Enter the cost matrix:\n");
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
            {
                cost[i][j] = sc.nextInt();
                if (cost[i][j] == 0)
                    cost[i][j] = 999;
            }
    }
    static void Prims()
    {
        int visited[] = new int[10];
        int ne = 1, i, j, min, a = 0, b = 0, u = 0, v = 0;
        int mincost = 0;
        visited[1] = 1;
        while (ne < n)
        {
            for (i = 1, min = 999; i <= n; i++)
                for (j = 1; j <= n; j++)
                    if (cost[i][j] < min)
                        if (visited[i] != 0)
                        {
                            min = cost[i][j];
                            a = u = i;
                            b = v = j;
                        }
            if (visited[u] == 0 || visited[v] == 0)
            {
                System.out.println("Edge" + ne++ + " : (" + a + ", " + b + ") " + "cost:" + min);
                mincost += min;
                visited[b] = 1;
            }
        }
    }
}
```

```

        cost[a][b] = cost[b][a] = 999;
    }
    System.out.println("Minimum cost: " + mincost);
}
}

```

OUTPUT

RUN 1	RUN 2	RUN 3																								
Enter the number of nodes: 6 Enter the cost matrix: <table><tr><td>0</td><td>3</td><td>999</td><td>999</td></tr><tr><td>3</td><td>0</td><td>1</td><td>999</td></tr><tr><td>999</td><td>1</td><td>0</td><td>6</td></tr><tr><td>999</td><td>999</td><td>6</td><td>0</td></tr><tr><td>6</td><td>999</td><td>999</td><td>8</td></tr><tr><td>5</td><td>4</td><td>4</td><td>5</td></tr></table> Edge1:(1,2)cost:3 Edge2:(2,3)cost:1 Edge3:(2,6)cost:4 Edge4:(6,5)cost:2 Edge5:(6,4)cost:5 Minimum cost: 15	0	3	999	999	3	0	1	999	999	1	0	6	999	999	6	0	6	999	999	8	5	4	4	5		
0	3	999	999																							
3	0	1	999																							
999	1	0	6																							
999	999	6	0																							
6	999	999	8																							
5	4	4	5																							

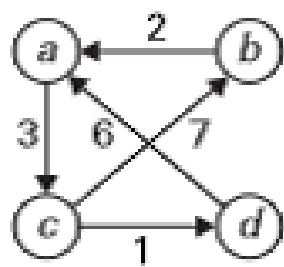
Program 17

Write Java programs to Implement All-Pairs Shortest Paths problem using Floyd's algorithm.

Solution :

```
import java.util.Scanner;
public class FloydsClass
{
    static final int MAX = 20; // max. size of cost matrix
    static int a[][]; // cost matrix
    static int n;
    // actual matrixsize
    public static void main(String args[])
    {
        a = new int[MAX][MAX];
        ReadMatrix();
        Floyds(); // find all pairs shortest path
        PrintMatrix();
    }
    static void ReadMatrix()
    {
        System.out.println("Enter the number of vertices");
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        System.out.println("Enter the Cost Matrix (999 for infinity)");
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                a[i][j] = sc.nextInt();
            }
        }
        sc.close();
    }
    static void Floyds()
    {
        for (int k = 1; k <= n; k++)
        {
            for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
            if ((a[i][k] + a[k][j]) < a[i][j])
                a[i][j] = a[i][k] + a[k][j];
        }
    }
    static void PrintMatrix()
    {
        System.out.println("The All Pair Shortest Path Matrix is:");
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
                System.out.print(a[i][j] + "\t");
            System.out.println("\n");
        }
    }
}
```

OUTPUT



RUN 1	RUN 2	RUN 3
<pre>Enter the number of vertices 4 Enter the Cost Matrix (999 for infinity) 0 999 3 999 2 0 999 999 999 7 0 1 6 999 999 0 The All Pair Shortest Path Matrix is 0 10 3 4 2 0 5 6 7 7 0 1 6 16 9 0</pre>		

:

Program 18

Implement Travelling Sales Person problem using Dynamic programming.

Solution :

```
import java.util.Scanner;
public class TravSalesPerson
{
    static int MAX = 100;
    static final int infinity = 999;
    public static void main(String args[])
    {
        int cost = infinity;
        int c[][] = new int[MAX][MAX]; // cost matrix
        int tour[] = new int[MAX]; // optimal tour
        int n; // max.cities
        System.out.println("Travelling Salesman Problem using Dynamic
        Programming\n");
        System.out.println("Enter number of cities: ");
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt();
        System.out.println("Enter Cost matrix:\n");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
            {
                c[i][j] = sc.nextInt();
                if (c[i][j] == 0)
                    c[i][j] = 999;
            }
        for (int i = 0; i < n; i++)
            tour[i] = i;
        cost = tspdp(c, tour, 0, n); // print tour cost and tour
        System.out.println("Minimum Tour Cost: " + cost);
        System.out.println("\nTour:");
        for (int i = 0; i < n; i++)
        {
            System.out.print(tour[i] + " -> ");
        }
        System.out.println(tour[0] + "\n");
        sc.close();
    }
    static int tspdp(int c[][], int tour[], int start, int n)
    {
        int i, j, k;
        int temp[] = new int[MAX];
        int mintour[] = new int[MAX];
        int mincost, cost;
        if (start == n - 2)
            return c[tour[n - 2]][tour[n - 1]] + c[tour[n - 1]][0];
        mincost = infinity;
        for (i = start + 1; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                temp[j] = tour[j];
                temp[start + 1] = tour[i];
                temp[i] = tour[start + 1];
```

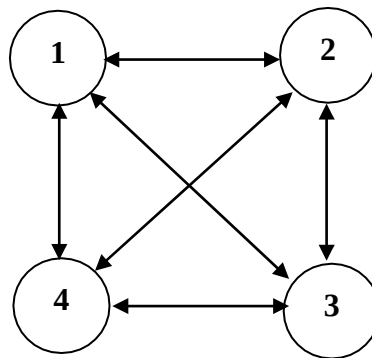
```

        if (c[tour[start]][tour[i]] + (cost = tspdp(c, temp, start +
1, n)) < mincost)
        {
            mincost = c[tour[start]][tour[i]] + cost;
            for (k = 0; k < n; k++)
                mintour[k] = temp[k];
        }
    }
    for (i = 0; i < n; i++)
        tour[i] = mintour[i];
    return mincost;
}

```

}

OUTPUT



RUN 1

Travelling Salesman Problem using Dynamic Programming

Enter number of cities:

4

Enter Cost matrix:

```

0    10   15   20
5    0    9   10
6    13   0   12
8    8    9    0

```

Minimum Tour Cost: 35

Tour:

0 -> 1 -> 3 -> 2 -> 0

RUN 2

RUN 3

Program 19

Design and implement Java Program to find the transitive closure using Warshal's algorithm.

```
import java.util.Scanner;

public class WarshallAlgorithm {

    // Function to perform Warshall's algorithm
    public static void transitiveClosure(int[][] graph) {
        int V = graph.length;
        int[][] tc = new int[V][V];

        // Initialize the transitive closure matrix
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                tc[i][j] = graph[i][j];
            }
        }

        // Update the transitive closure matrix
        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    tc[i][j] = tc[i][j] | (tc[i][k] & tc[k][j]);
                }
            }
        }

        // Print the transitive closure matrix
        System.out.println("Transitive closure matrix:");
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                System.out.print(tc[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Read number of vertices
    System.out.print("Enter the number of vertices: ");
    int V = scanner.nextInt();

    // Read the adjacency matrix
    int[][] graph = new int[V][V];
    System.out.println("Enter the adjacency matrix:");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            graph[i][j] = scanner.nextInt();
        }
    }

    // Compute and print transitive closure
    transitiveClosure(graph);

    scanner.close();
}

Enter the number of vertices: 4
Enter the adjacency matrix:
0 1 0 1
0 0 1 0
0 0 0 1
0 0 0 0
Transitive closure matrix:
1 1 1 1
0 1 1 1
0 0 1 1
0 0 0 1

```

Program 20

Design and implement Java Program to solve 0/1 Knapsack problem using Dynamic Programming method.

a)Knapsack Dynamic Programming

Solution :

```
import java.util.Scanner;
public class KnapsackDP
{
    static final int MAX = 21; // max. no. of objects
    static int w[]; // weights 0 to n-1
    static int p[]; // profits 0 to n-1
    static int n; // no. of objects
    static int M; // capacity ofKnapsack
    static int V[][]; // DP solution process - table
    static int Keep[][]; // to get objects in optimal solution

    public static void main(String args[])
    {
        w = new int[MAX];
        p = new int[MAX];
        V = new int [MAX][MAX];
        Keep = new int[MAX][MAX];
        int optsoln;
        ReadObjects();
        for (int i = 0; i <= M; i++)
            V[0][i] =0;
        for (int i = 0; i <= n; i++)
            V[i][0] =0;
        optsoln = Knapsack();
        System.out.println("Optimal solution = " + optsoln);
    }
    static int Knapsack()
    {
        int r; // remaining Knapsack capacity
        for (int i = 1; i <= n;i++)
            for (int j = 0; j <= M;j++)
                if ((w[i] <= j) && (p[i] + V[i - 1][j - w[i]] > V[i - 1][j]))
                {
                    V[i][j] = p[i] + V[i - 1][j - w[i]];
                    Keep[i][j] = 1;
                }
                else
                {
                    V[i][j] = V[i - 1][j];
                    Keep[i][j] = 0;
                }
        r = M; // Find the objects included in the Knapsack
        System.out.println("Selected Items = ");
    }
}
```

```

        for (int i = n; i > 0; i--) // start from Keep[n,M]
        if      (Keep[i][r] == 1)
        {
            System.out.println(i + " ");
            r = r - w[i];
        }
        System.out.println();
        return V[n][M];
    }

    static void ReadObjects()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Knapsack Problem - Dynamic Programming Solution: ");
        System.out.println("Enter the max capacity of knapsack:");
        M =sc.nextInt();
        System.out.println("Enter number of objects:");
        n =sc.nextInt();
        System.out.println("Enter Weights: ");
        for (int i = 1; i <= n;i++)
        w[i] = sc.nextInt();
        System.out.println("Enter Profits: ");
        for (int i = 1; i <= n;i++)
        p[i] = sc.nextInt();
        sc.close();
    }
}

```

OUTPUT

RUN 1	RUN 2	RUN 3
<pre> Knapsack Problem - Dynamic Programming Solution: Enter the max capacity of knapsack: 5 Enter number of objects: 4 Enter Weights: 2 1 3 2 Enter Profits: 12 10 20 15 Selected Items = 4 2 1 Optimal solution = 37 </pre>		

Program 21

Design and implement in Java to find a subset of a given set $S = \{S_1, S_2, \dots, S_n\}$ of n positive integers whose SUM is equal to a given positive integer d . For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$, there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. Display a suitable message, if the given problem instance doesn't have a solution.

Solution :

```
import java.util.Scanner;
public class SumOfSubsets
{
    int[]w;
    int[] x;
    intsum;
    int total = 0;
    public void process()
    {
        getData();
        System.out.println("The subsets are:");
        subset(0, 1, total);
    }
    private void getData()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements:");
        int n = sc.nextInt();
        w = new int[n + 1];
        x = new int[n + 1];
        System.out.println("Enter " + n + " Elements in ascending order :");
        for (int i = 1; i < n + 1; i++)
        {
            w[i] = sc.nextInt();
            total += w[i];
        }
        System.out.println("Enter the sum to be obtained: ");
        sum = sc.nextInt();
        if (total < sum || sum < w[1])
        {
            System.out.println("No possible subsets!!!");
            System.exit(1);
        }
    }
}
```

```

        sc.close();
    }
    private void subset(int s, int k, int r)
    {
        int i = 0;
        x[k] = 1;
        if (s + w[k] == sum)
        {
            System.out.print("(");
            for (i = 1; i <= k; i++)
            {
                if(x[i]==1)
                    System.out.print(" " + w[i]);
            }
            System.out.println(")");
        }
        else if ((s + w[k] + w[k + 1]) <= sum)
        {
            subset(s + w[k], k + 1, r - w[k]);
        }
        if ((s + r - w[k]) >= sum && (s + w[k + 1]) <= sum)
        {
            x[k] = 0;
            subset(s, k + 1, r - w[k]);
        }
    }
}
public static void main(String[] args)
{
    new SumOfSubsets().process();
}
}

```

OUTPUT

RUN 1	RUN 2	RUN 3
<pre> Enter the number of elements Enter 5 Elements in ascending order 5 10 15 20 Enter the sum to be obtained 30 The subsets are: (5 10 15) (5 25) (10 20) </pre>		

Program 22

Design and implement Java Program for N Queen's problem using Backtracking.

```
import java.util.Scanner;

public class NQueens {
    private int N;
    private int[][] board;

    public NQueens(int N) {
        this.N = N;
        board = new int[N][N];
    }

    public boolean solve() {
        if (!solveNQueens(0)) {
            System.out.println("No solution exists");
            return false;
        }
        printSolution();
        return true;
    }

    private boolean solveNQueens(int row) {
        if (row >= N) {
            return true;
        }

        for (int col = 0; col < N; col++) {
            if (isSafe(row, col)) {
                board[row][col] = 1;
                if (solveNQueens(row + 1)) {
```

```

        return true;
    }
    board[row][col] = 0; // backtrack
}
}
return false;
}

private boolean isSafe(int row, int col) {
    // Check this column on upper side
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 1) {
            return false;
        }
    }

    // Check upper diagonal on left side
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1) {
            return false;
        }
    }

    // Check upper diagonal on right side
    for (int i = row, j = col; i >= 0 && j < N; i--, j++) {
        if (board[i][j] == 1) {
            return false;
        }
    }

    return true;
}

```

```

private void printSolution() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            System.out.print((board[i][j] == 1 ? "Q " : ".
"));
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter the number of queens: ");
    int N = scanner.nextInt();

    NQueens queens = new NQueens(N);
    queens.solve();

    scanner.close();
}
}

```

Enter the number of queens: 4

```

.   Q   .   .
.   .   .   Q
Q   .   .   .
.   .   Q   .

```

CHAPTER 4

VIVA QUESTIONS AND ANSWERS

1. What is an algorithm?

An algorithm is a sequence of finite number of unambiguous instructions for solving a problem. i.e., for obtaining a required output for any legitimate input in a finite amount of time.

2. Name some basic Efficiency classes.

Constant, Logarithmic, Linear, $n \log n$, Quadratic, Cubic, Exponential, Factorial

3. What are algorithm design techniques?

Algorithm design techniques (or strategies or paradigms) are general approaches to solving problems algorithmically, applicable to a variety of problems from different areas of computing. General design techniques are:

- | | | |
|----------------------|--------------------------|----------------------------|
| (i) Brute force | (ii) divide and conquer | (iii) Decrease and conquer |
| (i) Greedy technique | (vi) dynamic programming | (vii) Backtracking |
| | (viii) branch and bound | |

4. How is the efficiency of the algorithm defined?

The efficiency of an algorithm is defined with the components.

- Time efficiency -indicates how fast the algorithm runs
- Space efficiency -indicates how much extra memory the algorithm needs

5. Write the Analysis for the Quicksort.

$O(n \log n)$ in average and best cases $O(n^2)$ in worst case

6. Is insertion sort better than the mergesort?

Insertionsortworksexceedinglyfastonarraysoflessthen16elements,though for large,, n its computing time is $O(n^2)$.

7. Specify the algorithms used for constructing Minimum cost spanning tree.

Prim's Algorithm, Kruskal's Algorithm

8. Write the difference between the Greedy method and Dynamic programming.

Greedy method	Dynamic programming
1. Only one sequence of decision is generated.	1. Many number of decisions are generated.
It does not guarantee to give an optimal solution always.	2. It definitely gives an optimal solution always.

9. State the time efficiency of floyd's algorithm

$O(n^3)$. It is cubic.

10. Give the time complexity and space complexity of traveling salesperson problem.

Time complexity is $O(n^2 2^n)$.

Space complexity is $O(n 2^n)$.

11. What is the average case complexity for mergesort algorithm.

Time complexity is $O(n \log n)$

12. Quick sort uses _____

Divide and Conquer
Technique

13. What is the best case complexity for insertion sort

Time complexity is $O(n)$

14. Which of the following standard algorithms is not Dynamic Programming based.

- Bellman–Ford Algorithm for single source shortest path
- Floyd Warshall Algorithm for all pairs shortest paths
- Knapsack problem
- Prim's Minimum Spanning Tree
- Prim's Minimum Spanning Tree is a Greedy Algorithm. All other are dynamic programming based.

15. The recurrence relation capturing the optimal time of the Tower of Hanoi problem with n discs is.

- (a) $t(n) = 2t(n - 2) + 2$
- (b) $t(n) = 2t(n - 1) + n$
- (c) $t(n) = 2t(n/2) + 1$
- (d) $t(n) = 2t(n - 1) + 1$**

16. Define spanning tree

A spanning tree of a connected graph is its connected acyclic subgraph (i.e a tree) that contains all vertices of a graph.

17. Define Minimum Spanning tree.

A Minimum Spanning tree of a weighted connected graph is its spanning tree of the smallest weight, where weight of the tree is defined sum of the weights on all its edges.

18. Define Kruskal's Algorithm

Kruskal's algorithm looks at a minimum spanning tree for a weighted connected graph $G = (V, E)$ as an acyclic subgraph with $|V| - 1$ edges for which the sum of the edge weights is the smallest.

19. Define Prim's Algorithm

Prim's algorithm is a greedy algorithm for constructing a minimum spanning tree of a weighted connected graph. It works by attaching to a previously constructed subtree a vertex to the vertices already in the tree.

20. Explain Warshall's algorithm

Warshall's algorithm constructs the transitive closure of a given digraph with n vertices through a series of n by n Boolean matrices $R(0), \dots, R(k-1), R(k), \dots, R(n)$. Each of these matrices provides certain information about directed paths in the digraph.

21. Explain All-pair shortest-paths problem

Given a weighted connected graph (undirected or directed), the all pairs shortest paths problem asks to find the distances (the lengths of the shortest path) from each vertex to all other vertices.

22. Explain Knapsack problem

Given n items of known weights $w_1, w_2, \dots, w_n$ and values $v_1, v_2, \dots, v_n$ and a knapsack of capacity W , find the most valuable subset of the items that fit into the knapsack. (Assuming all the weights and the knapsack's capacity are positive integers the item values do not have to be integers.)

23. Explain "Traveling salesman problem"?

A salesman has to travel n cities starting from any one of the cities and visit the remaining cities exactly once and come back to the city where he started his journey in such a manner that either the distance is minimum or cost is minimum. This is known as traveling salesman problem.

24. Explain Backtracking

The principal idea is to construct solutions one component at a time and evaluate such partially constructed candidates as follows.

- If a partially constructed solution can be developed further without violating the problem's constraints, it is done by taking the first remaining legitimate option for the next component.
- If there is no legitimate option for the next component, no alternatives for any remaining component need to be considered. In this case, the algorithm backtracks to replace the last component of the partially constructed solution with its next option.

25. Explain promising and nonpromising node.

A node in a state space tree is said to be promising if it corresponds to a partially constructed solution that may still lead to a complete solution; otherwise it is called nonpromising.

26. Explain n-Queens problem.

The problem is to place n queens on an n by n chessboard so that no two queens attack each other by being in the same row or same column or on the same diagonal.

27. Explain Subset-Sum Problem.

We consider the subset-sum problem: Find a subset of a given set $S = \{S_1, S_2, \dots, S_n\}$ of n positive integers whose sum is equal to a given positive integer d .

28. Explain Knapsack Problem.

Find the most valuable subset of n items of given positive integer weights and values that fit into a knapsack of a given positive integer capacity.

29. Define order of growth.

The efficiency analysis framework concentrates on the order of growth of an algorithm's basic operation count as the principal indicator of the algorithm's efficiency. To compare and rank such orders of growth we use three notations

- O (Big Oh) notation
- Ω (Big Omega) notation
- Θ (Big Theta) notation

30. What is the use of Asymptotic Notations?

The notations O , Ω and Θ are used to indicate and compare the asymptotic orders of growth of functions expressing algorithm efficiencies.

APPENDIX

Time Complexity of Sorting Algorithms

Algorithm Name	Average	Best	Worst	Space	Stability	Remarks
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	Stable	Always use a modified bubble sort
Modified Bubble sort	$O(n^2)$	$O(n)$	$O(n^2)$	Constant	Stable	Stops after reaching a sorted array
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Constant	Stable	Even a perfectly sorted input requires scanning the entire array
Insertion Sort	$O(n^2)$	$O(n)$	$O(n^2)$	Constant	Stable	In the best case (already sorted), every insert requires constant time
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Constant	Instable	By using input array as storage for the heap, it is possible to achieve constant space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Depends	Stable	On arrays, merge sort requires $O(n)$ space; on linked lists, mergesort requires constant space
Quicksort	$O(n \log n)$	$1.38n \log n$	$O(n^2)$	Constant	Stable	Randomly picking a pivot value (or shuffling the array prior to sorting) can help avoid worst case scenarios such as a perfectly sorted array.